

AWS Bastion Host Architecture Lab — Full Guide

Made by Harsh Kumar | GitHub: github.com/HARSH-dev1708

This lab builds the exact architecture in your diagram: a VPC with a **public subnet** holding a **Bastion Host**, and a **private subnet** holding two **App Instances**. You SSH from your laptop into the Bastion, and from the Bastion you SSH onward into the private App Instances. The "SSH forwarding on your local system" your instructor mentioned is the technique that lets you do that second hop *without* copying your private key onto the Bastion.

1. The Concepts Behind This Architecture

Before touching the console, it's worth understanding *why* this design exists, because every piece in the diagram solves a specific security problem.

VPC (Virtual Private Cloud) is your own isolated network inside AWS — a private slice of the cloud with its own IP address range (CIDR block), where you control subnets, routing, and security. Nothing outside it can reach inside unless you explicitly allow it.

Subnets split the VPC into smaller network segments, each tied to one Availability Zone. A subnet's "public" or "private" label isn't a real AWS setting — it's just a convention based on whether that subnet's route table sends internet-bound traffic to an Internet Gateway. A **public subnet** has a route to an Internet Gateway, so instances in it *can* get public IPs and be reached from the internet. A **private subnet** has no such route, so nothing on the internet can initiate a connection to instances inside it — that's exactly where you want sensitive app servers to live.

Internet Gateway (IGW) is the door between your VPC and the public internet. It's attached to the VPC once, and only subnets whose route table points `0.0.0.0/0` at it actually behave as "public."

NAT Gateway solves a different problem: your private App Instances still need *outbound* internet access sometimes (to download OS patches, pull packages, etc.) without being reachable *inbound* from the internet. A NAT Gateway sits in the public subnet, and the private subnet's route table sends its outbound traffic through it. Traffic can leave, but nothing from the internet can use the NAT Gateway to get in. This is optional for the lab itself but good practice, so it's included below.

Made by Harsh Kumar | GitHub: github.com/HARSH-dev1708

Route Tables are the actual switches that decide whether a subnet is public or private. Each subnet is associated with one route table; the rules in that table determine where traffic goes.

Security Groups are stateful, instance-level firewalls. In your diagram there are two: one attached to the Bastion Host that only allows SSH (port 22) in from your IP, and one attached to the App Instances that only allows SSH in from the Bastion's security group — not from your IP directly, and not from the internet at all. This is the core of the bastion pattern: the App Instances trust *only* the Bastion.

Bastion Host (Jump Box) is the single, hardened, monitored entry point into your private network. Instead of giving every app server a public IP (a huge attack surface), you expose exactly one small instance, lock it down tightly, and route all administrative access through it.

SSH Forwarding is the mechanism that makes the "Bastion → App Instance" hop work securely. There are two common approaches, and your instructor's phrase "do SSH forwarding on your local system" almost certainly means **SSH Agent Forwarding**, so that's the primary method below — but I'll also show the modern alternative (**ProxyJump**), which most engineers actually prefer today, so you understand both.

- **SSH Agent Forwarding (`ssh -A`)**: Your private key *never leaves your laptop*. When you SSH into the Bastion with agent forwarding enabled, your laptop's `ssh-agent` is "forwarded" over that connection. When you then SSH from the Bastion to an App Instance, the Bastion asks your laptop's agent to sign the authentication challenge on the fly. The key material is never copied or stored on the Bastion — only a live, temporary access to the agent's signing ability.
- **ProxyJump (`ssh -J`)**: A single command on your laptop tunnels straight through the Bastion to the App Instance in one shot, without you ever getting a separate shell on the Bastion in between. This avoids the (minor) security caveat of agent forwarding, where anyone with root on the Bastion could technically request a signature from your agent while you're connected.

You'll use both so you can compare them.

2. Architecture Summary (what we're building)

Component	Detail
VPC	<code>10.0.0.0/16</code>
Public Subnet	<code>10.0.1.0/24</code> — hosts Bastion
Private Subnet	<code>10.0.2.0/24</code> — hosts App-Instance-1 and App-Instance-2

Internet Gateway	attached to VPC, routed from public subnet
NAT Gateway	in public subnet, routed from private subnet (optional but recommended)
Security Group: bastion-sg	allow SSH (22) from My IP only
Security Group: app-sg	allow SSH (22) from bastion-sg only
Key pair	one .pem key, used for all three instances

3. Prerequisites

You'll need an AWS account with console access, and a terminal on your laptop (built-in on Mac/Linux; on Windows use PowerShell, WSL, or Git Bash — all support OpenSSH natively now).

4. Step-by-Step: Building the Network

Step 1 — Create the VPC

Go to **VPC console** → **Your VPCs** → **Create VPC**.

- Choose "VPC only."
- Name: **lab-vpc**
- IPv4 CIDR: **10.0.0.0/16**
- Leave tenancy as default, create.

Step 2 — Create the Subnets

VPC console → **Subnets** → **Create subnet**, selecting **lab-vpc** as the parent VPC, and create two:

- **public-subnet** — CIDR **10.0.1.0/24** — pick an AZ (e.g. **us-east-1a**)
- **private-subnet** — CIDR **10.0.2.0/24** — same AZ for simplicity

Step 3 — Create and Attach an Internet Gateway

VPC console → **Internet Gateways** → **Create internet gateway**, name it `lab-igw`, create it, then select it and **Actions** → **Attach to VPC** → `lab-vpc`.

Step 4 — Create Route Tables

VPC console → **Route Tables** → **Create route table**:

- `public-rt`, associate with `lab-vpc`. After creating, edit its **Routes** tab → add route `0.0.0.0/0` → target `lab-igw`. Then go to the **Subnet associations** tab → associate `public-subnet`.
- `private-rt`, associate with `lab-vpc`. Associate it with `private-subnet`. Leave its default local route only for now (you'll add the NAT route in Step 6 if you choose to set up NAT).

Step 5 — Enable Auto-assign Public IP on the Public Subnet

Select `public-subnet` → **Actions** → **Edit subnet settings** → enable "Auto-assign public IPv4 address." This way your Bastion automatically gets a public IP at launch without you doing it manually.

Step 6 — (Optional but recommended) Create a NAT Gateway

VPC console → **NAT Gateways** → **Create NAT gateway**:

- Subnet: `public-subnet` (NAT Gateways must live in a public subnet)
- Allocate an Elastic IP for it
- Create it, wait until it's "Available"

Then edit `private-rt` → add route `0.0.0.0/0` → target the new NAT Gateway. Now your private instances can reach the internet outbound (for `yum update`, etc.) but remain unreachable inbound.

5. Step-by-Step: Security Groups

Step 7 — Bastion Security Group

EC2 console → **Security Groups** → **Create security group**:

- Name: `bastion-sg`, VPC: `lab-vpc`
- Inbound rule: Type SSH, Port 22, Source **My IP** (AWS auto-fills your current public IP — this is far safer than `0.0.0.0/0`, which exposes SSH to the entire internet)

Made by Harsh Kumar | GitHub: github.com/HARSH-dev1708

- Outbound: leave default (allow all)

Step 8 — App Instance Security Group

Create another security group:

- Name: `app-sg`, VPC: `lab-vpc`
- Inbound rule: Type SSH, Port 22, **Source:** select `bastion-sg` (not an IP — you're naming the other security group directly, so only traffic originating from instances in `bastion-sg` is allowed in)
- Outbound: leave default

This single rule is what enforces the entire bastion pattern at the network level: even if someone had network access to the private subnet's CIDR range some other way, they still couldn't SSH into the app instances unless that traffic is tagged as coming from the bastion's security group.

6. Step-by-Step: Launching the Instances

Step 9 — Create / select a Key Pair

EC2 console → Key Pairs → Create key pair:

- Name: `lab-key`
- Type: RSA (or ED25519, either works fine with OpenSSH)
- Format: `.pem` (for Mac/Linux/WSL) or `.ppk` if you're using PuTTY
- Download it and keep it somewhere safe — AWS does not let you re-download it later.

On your local machine, lock down its permissions (required by SSH or it will refuse to use the key):

```
chmod 400 lab-key.pem
```

Step 10 — Launch the Bastion Host

EC2 console → Launch instance:

- Name: `BastionHost`
- AMI: Amazon Linux 2023 (or Ubuntu, your call)
- Instance type: `t2.micro` (free tier eligible)
- Key pair: `lab-key`

- Network settings → Edit: VPC `lab-vpc`, Subnet `public-subnet`, Auto-assign public IP: **Enable**
- Security group: select existing → `bastion-sg`
- Launch

Step 11 — Launch the App Instances

Repeat twice, naming them `App-Instance-1` and `App-Instance-2`:

- Same AMI, same key pair `lab-key`
- Subnet: `private-subnet`
- Auto-assign public IP: **Disable** (it must stay private — no public IP at all)
- Security group: `app-sg`

Once all three are running, note their **private IPv4 addresses** (`App-Instance-1`, `App-Instance-2`) and the **Bastion's public IPv4 address** from the EC2 console — you'll need all three for the SSH steps.

7. Step-by-Step: SSH Forwarding From Your Local Machine

This is the part your instructor specifically called out. Here are the two ways to do it — try both so you understand the tradeoff.

Method A — SSH Agent Forwarding (`ssh -A`)

First, load your key into your local SSH agent so it's available to forward:

```
eval "$(ssh-agent -s)"
ssh-add lab-key.pem
```

Now connect to the Bastion **with agent forwarding enabled** using the `-A` flag:

```
ssh -A -i lab-key.pem ec2-user@<BASTION_PUBLIC_IP>
```

You're now inside the Bastion. Notice you did **not** copy `lab-key.pem` onto it. From inside this Bastion shell, SSH onward to an App Instance using only its private IP — no `-i` flag needed, because the agent forwarded from your laptop handles authentication transparently:

```
ssh ec2-user@<APP_INSTANCE_1_PRIVATE_IP>
```

You should land on App-Instance-1 having never placed your private key on the Bastion at any point. This is exactly what "SSH forwarding on your local system" refers to — the forwarding happens because your *local* agent is the one doing the cryptographic signing for every hop.

Method B — ProxyJump (the modern one-liner)

This skips the separate Bastion shell entirely and tunnels straight through in a single command, run **only on your laptop**:

```
ssh -i lab-key.pem -J ec2-user@<BASTION_PUBLIC_IP>  
ec2-user@<APP_INSTANCE_1_PRIVATE_IP>
```

To make this permanent and avoid retyping IPs, add this to your local `~/.ssh/config`:

Host bastion

```
HostName <BASTION_PUBLIC_IP>  
User ec2-user  
IdentityFile ~/.ssh/lab-key.pem
```

Host app1

```
HostName <APP_INSTANCE_1_PRIVATE_IP>  
User ec2-user  
IdentityFile ~/.ssh/lab-key.pem  
ProxyJump bastion
```

Host app2

```
HostName <APP_INSTANCE_2_PRIVATE_IP>  
User ec2-user  
IdentityFile ~/.ssh/lab-key.pem  
ProxyJump bastion
```

After this, connecting is just:

```
ssh app1
```

SSH reads the config, opens a connection to `bastion` first, then tunnels the second connection through it automatically — same outcome as agent forwarding, but the key file reference is explicit instead of relying on a forwarded agent, and you never get an interactive shell on the Bastion at all unless you ask for one.

Made by Harsh Kumar | GitHub: github.com/HARSH-dev1708

8. Verifying the Architecture Works as Designed

Run these checks to prove each security boundary is actually enforced:

1. From your laptop, try to SSH **directly** to an App Instance's private IP — it should hang or time out, since private IPs aren't routable from outside the VPC and `app-sg` doesn't allow your IP anyway.
2. From the Bastion, run `ssh ec2-user@<App-Instance-2 private IP>` to confirm you can reach both app instances, demonstrating the one-to-many "fan out" shown in the diagram.
3. From an App Instance, try `curl -m 5 google.com` — if you set up the NAT Gateway in Step 6, this succeeds (outbound works); if you skipped it, this times out (no outbound route), which is also a valid lab configuration if your instructor only cares about the SSH topology.

9. Cleanup (do this after the lab to avoid charges)

In reverse order: terminate all three EC2 instances → delete the NAT Gateway (if created) → release its Elastic IP → delete the route tables you created → detach and delete the Internet Gateway → delete the subnets → delete the VPC. NAT Gateways and Elastic IPs are the two things most likely to quietly rack up cost if left running, so don't skip those.

10. Quick Recap of What You Learned

The lab demonstrates defense in depth through network segmentation (public vs. private subnets), least-privilege firewalling (security groups referencing each other instead of open CIDR ranges), and a single audited entry point (the bastion pattern) — combined with a credential-handling technique (SSH agent forwarding or ProxyJump) that lets you reach deep into the private network without ever letting your private key touch a machine you don't fully control. This exact pattern — bastion + private subnet + agent forwarding — is one of the most common real-world setups for securely administering cloud infrastructure.